

Received March 14, 2019, accepted April 28, 2019, date of publication May 8, 2019, date of current version June 7, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2915681

NeuralTaint: A Key Segment Marking Tool Based on Neural Network

XING ZHANG¹, CHAO FENG, RUNHAO LI¹, JING LEI¹, AND CHAOJING TANG

School of Electronic Science and Technology, National University of Defense Technology, Changsha 410073, China

Corresponding author: Chao Feng (chaofeng@nudt.edu.cn)

ABSTRACT Dynamic taint analysis techniques are a popular dynamic software analysis method. Marking a key segment of program function by dynamic taint analysis is an important part of software vulnerability research. Key segment marking usually related to the control flow taint analysis, however, several specific program structure may cause failure in key segment marking due to the control flow dependence, and overtainting and undertainting problem. In this paper, we proposed a novel method to mark a key segment accurately and efficiently with deep learning technology. Firstly, we fit the program function execution into a continuous function by the convolutional network, and then mark the key segment roughly through derivative information of fitted neural network. Finally, we mark the key segment of specific program function completely and accurately by filtering and diffusion algorithm. We developed the key segment marking tool NeuralTaint on this principle. We design an experiment to select the specific neural network structure of NeuralTaint. Our extensive evaluations demonstrate that NeuralTaint significantly outperforms the two state-of-the-art traditional dynamic taint analysis tool on seven popular real-world programs.

INDEX TERMS Taint analysis, symbolic execution, software vulnerability, neural network, key segment marking, deep learning.

I. INTRODUCTION

Dynamic analysis techniques have increasingly occupying an important position in software vulnerability analysis, and one of the key issues is to analyze the impacts of input on program functionalities. Program input is analyzed thoroughly to diagnose which part affects the execution of function [1]. Based on the analysis, researchers could further explore the structure of the program or generate test cases with high path coverage. In this paper, key segment in the program input refers to the part of input which could directly affect the execution of certain function of program [2]. To mark key segment of program input accurately, current approaches mainly depend on dynamic taint analysis and dynamic symbolic execution [3]. Dynamic taint analysis runs a program and observes which are affected by predefined taint sources such as user input [1]. Dynamic symbolic execution builds a logical formula describing the relationship between the program execution path and the program input [4].

However, both dynamic taint analysis and dynamic symbolic execution suffer a lot of problems. Some particular program structure can lead to the failure of the taint

propagation policy and symbolic constraint collection. Specifically, dynamic taint analysis may suffer from problems of taint addresses, control-flow taint, overtainting, and undertainting [5] while dynamic symbolic execution faces the problems such as control-flow dependence and symbolic jumps [6], which could result in the inability to achieve accurate key segment identifying when analyzing binary programs. Given limitations of traditional techniques, we investigate an alternative line of research, which uses the neural networks to obtain the relationship between the program input with the program function execution, and further mark the key segment of input according to the derivative information. With continuous development of computer hardware, the neural networks, represented by deep learning methods, can learn and express complex deep logical relationships [7], thus been widely applied in many fields. Therefore, this paper leverages neural networks to map the program input to program execution, instead of the exploration to internal programs with taint policy and symbolic constraint, which could effectively avoid the inaccurate diagnosing problems using traditional dynamic analysis techniques.

The problem of key segment marking based on neural network can be divided into two parts, the first of which is how to achieve the continuous and smooth fitting from

The associate editor coordinating the review of this manuscript and approving it for publication was Aniello Castiglione.

program input to program execution with neural networks, and the second is how to locate key segment with well-fitted neural networks. In this paper, we construct a variety of convolutional networks based on real-valued/one-hot vector input modeling and classification/regression network for the mapping of program input to function execution. We demonstrate the effectiveness of locating key segment with derivative and design an algorithm to achieve accurate key segment marking. Related experiments were designed and conducted to explore the effects of different input modeling, network structure, and network parameters on fitting accuracy, fitting speed, and marking accuracy of key segment. Based on experimental results and principles analysis, we found that one-hot input modeling can not only obtain a model with higher fitting accuracy, but also improve the efficiency and accuracy of the key segment marking algorithm effectively since the derivative contains more information. Meanwhile, regression network structure can fit faster with less overfitting.

In this paper, we combine the convolutional regression network with one-hot vector input modeling, and implement the key segment marking tool NeuralTaint. We select 7 real-world applications under Linux platform and compare with 2 taint analysis tools. The experimental results show that NeuralTaint consistently outperforms all the other traditional dynamic analysis tools by a wide margin both in term of accuracy and program structure applicability.

In summary, the main contributions in this paper are listed as follows:

1) We reasoning the feasibility of marking key segment through derivative information with mathematical theory. And introduce several methods to fit the execution of program function into continuous, smooth and trainable function by neural networks.

2) We design convolutional network structures with different input modeling methods, network types, and network parameters to fit the execution of program function. Our method becomes the first to compare the difference between the fitting rate and the fitting speed under different network structures through experiment. We select the network structure which has best performance as the fitting network of NeuralTaint.

3) We propose a key segment marking algorithm based on the derivative information of program input and develop key segment marking tool NeuralTaint.

4) We evaluate our method by comparing with traditional taint analysis tools. This method proposed in the paper has higher marking accuracy and faster executing speed, which can effectively overcome the problems of control flow dependence, and symbolic address in traditional approaches.

This paper is organized as follows: Section 2 presents theoretical basis on key segment marking through neural network. Section 3 introduces the overview of key segment marking method and Section 4 describes implementation of NeuralTaint in detail. Section 5 verifies the effectiveness of NeuralTaint under experiment. Section 6 summarizes the related work and Section 7 concludes the paper.

II. THEORETICAL BASIC

This section firstly explains the rationality of determining the position of key segment by derivative, and then introduces several techniques of fitting method.

A. MATHEMATICAL BASIC

Suppose a binary program without random function can be regarded as a multivariate function $\vec{y} = G(\vec{x})$, where $\vec{x}$ is the program input vector and $\vec{x} \in \{x^i | 0 \leq i < n, x^i \in \mathbb{N}, x^i \in [0, 0xFF]\}$, where n is the length in bytes of $\vec{x}$, and $\vec{y}$ is the various forms output of the program. In order to explore the effect of $\vec{x}$ on a function f inside $G(\vec{x})$, suppose a function $\vec{y} = F(\vec{x})$, where $\vec{y} \in \{y_i | i, y^i \in \mathbb{N}, 0 \leq y_i, 0 \leq i \leq n\}$ and $\vec{y}$ is used to indicate the number of times f is executed. At this time, $F(\vec{x})$ is consistent with $G(\vec{x})$ in the processing of f . Therefore, looking for the key segment in $\vec{x}$ that has the greatest influence on the execution of the program function f depends on the positions in the component of $\vec{x}$ that contribute the most to the change of $\vec{y}$.

Assuming that $F(\vec{x})$ is derivable at $\vec{x}_0$, the first-order Taylor expansion of $F(\vec{x})$ at input $\vec{x}_0$ as in (1):

$$F(x^0, x^1, \dots, x^{n-1}) = F(x_0^0, x_0^1, \dots, x_0^{n-1}) + \sum_{i=0}^{n-1} (x^i - x_0^i) F'(x_0^0, x_0^1, \dots, x_0^{n-1}) + o^2 \quad (1)$$

where o^2 is a second-order infinitesimal remainder, so we can derive formula as in (2):

$$F(x^0, x^1, \dots, x^{n-1}) - F(x_0^0, x_0^1, \dots, x_0^{n-1}) \sim \sum_{i=0}^{n-1} (x^i - x_0^i) F'(x_0^0, x_0^1, \dots, x_0^{n-1}) \quad (2)$$

It can be seen that for the $\vec{x}_0$ of $\vec{y} = F(\vec{x})$, if the partial derivative $\partial F(x_0^i) / \partial x_0^i$ at x_0^i reaches the maximum value, then we can say x_0^i contributes the most to the change of $\vec{y}$, which means that x_0^i , the i -th component of x_0 is the key segment of the execution of f .

However, $F(\vec{x})$ is usually a discrete, non-transducible function. It is not possible to directly determine the position of the key segment through derivation. For this case, we construct a continuous derivable function $\vec{y} = F(\vec{x}, \theta)$ to fit $\vec{y} = F(\vec{x})$, where θ is the parameter to be trained. We collect m program inputs as training data $\vec{X}$, execute them and obtain the execution times of f as label data $\vec{Y}$. As long as the appropriate parameter θ is found to achieve accurate mapping from input sample $\vec{X}$ to output sample $\vec{Y}$, we can use $\vec{y} = F(\vec{x}, \theta)$ instead of $\vec{y} = F(\vec{x})$ to locate the key segment of the execution of f .

The key problem is the construction method of $\vec{y} = F(\vec{x}, \theta)$ and the training method of the parameter θ . The construction problem of the function $\vec{y} = F(\vec{x}, \theta)$ is a regression analysis problem. Regression analysis is a set of statistical processes for estimating the relationships among variables in statistical modeling [8]. Currently methods used in regression analysis are linear regression, logistic regression, polynomial regression, softmax regression, and neural network regression [9].

This paper selects softmax regression and neural network regression method which are better for multivariate nonlinear fitting.

B. MULTI-CLASS LOGISTIC REGRESSION

Multi-class logistic regression is a classification method that generalizes logistic regression to multi-class problems [10]. This fitting method has now been widely applied to multi-classification problems. The fitting function is $\vec{y} = F_m(\vec{x}, \theta)$, where $\vec{y} \in \{y_i | i, y^i \in \mathbb{N}, 0 \leq y_i \leq 1, 0 \leq i \leq n\}$, and each component output of the function is 0 or 1. So before applying the fitting method, we need to convert the execution times of target function $\vec{y} \in \{y_i | y^i \in \mathbb{N}, 0 \leq y_i\}$ into the vector of $\{0, 1\}^K$, that is, we can regard the output component y_i as a one-hot vector that is divided into K classes at most. The output for classifying $\vec{x}$ into category k for K categories is as in (3), where θ is the function parameter to be trained.

$$F_m(y = k | x, \theta) = \frac{e^{\theta_k^T x}}{\sum_{t=1}^{K-1} e^{\theta_t^T x}}, \quad k = 1, 2, \dots, K-1 \quad (3)$$

Multi-class logistic regression usually adopts the cross entropy function as the loss function [11]. For m samples dataset, the loss function can be expressed as in (4), where $F_{m,i,j}$ represents the probability that the i -th sample belongs to category j .

$$L(\theta) = -\frac{1}{m} \sum_i^m \sum_j^K 1\{j = y^i\} \log(F_{m,i,j}) \quad (4)$$

The fitting parameter θ can be derived by using the gradient descent method or the newton iteration method for the loss function. With the continuous development of deep learning technology, the current multi-class logistic regression is usually implemented by means of neural network, that is, the gradient descent method is generally used to derive the fitting parameter θ [10].

C. NEURAL NETWORK REGRESSION

Neural network regression has strong versatility, which can fit all functions theoretically [12], [13]. The network could be represented diversely with real-valued vector as output, and the modeling methods varies from different specific problems. In general, the neural network consists of a linear function $y = F_L(x, \theta) = \sum \theta x + b$ and an activation function $y = F_a(x, \theta)$, while the linear function $y = F_L(x, \theta)$ through different configurations form a different network $y = F_s(x, \theta)$, whose outputs act as an input to the activation function which is the output of the fitting function. At this time, the neural network regression function is $\vec{y} = F_N(\vec{x}, \theta) = F_a(F_s(\vec{x}, \theta_1), \theta_2)$. The loss functions can also vary with the network model, and the most common loss function is the mean squared error (MSE) [14]. For m data samples, the MSE function can be expressed as in (5), where y_i represents the true value of the i -th sample, and y_i' represents the fitted

value of the i -th sample calculated through $\vec{y} = F_N(\vec{x}, \theta)$.

$$L(\theta) = -\frac{1}{m} \sum_i^m (y_i - y_i')^2 \quad (5)$$

For neural network fitting, the fitting parameter θ can be derived from the loss function using the gradient descent method [12], [13].

D. KEY SEGMENT MARKING METHOD BASED ON FITTED FUNCTION

If we use multi-class logistic regression to achieve function fitting, then one-hot modeling is urgent for output $\vec{y}$, which means converting $\vec{y} \in \{y_i | i, y^i \in \mathbb{N}, 0 \leq y_i, 0 \leq i \leq n\}$ to $\vec{y} \in \{y_i | i, y^i \in \mathbb{N}, 0 \leq y_i \leq 1, 0 \leq i \leq n\}$; If we use neural network regression for function fitting, $\vec{y}$ is transferred from integer to real-valued vector, that is $\vec{y} \in \{y_i | i \in \mathbb{N}, y^i \in \mathbb{R}, y_i \leq N, 0 \leq i \leq n\}$. More specific modeling techniques within related advantages and disadvantages of the two fitting methods will be introduced and explained in section 4.

Base on the fitted function $\vec{y} = F(\vec{x}, \theta)$, we can mark the key segment of target function through the partial derivative information of $\vec{y}$ to $\vec{x}$. The i -th component of output $\vec{y}$ partial derivative to each component j of input $\vec{x}_0$ is $G_i^j = \partial y_i / \partial x_0^j$. We can get $G_i^j = \partial F(\vec{x}_0, \theta) / \partial x_0^j$. The larger the G_i^j is, the higher probability of the x_0^j is key segment. Base on this, we can mark the key segment by sorting partial derivative G_i^j of all components j of $\vec{x}_0$ and selecting the largest ones.

III. OVERVIEW OF OUR APPROACH

This section introduces the overall idea of key segment marking method based on neural networks. Firstly, the mapping of program input to the execution times of target program function is fitted into a continuous derivable function through the neural network, and then we leverage the derivative information of fitted functions to mark the key segment roughly. Eventually, we implement the key segment marking accurately by filtering the coarse key segments and diffusion algorithm. The overall design is shown in the Figure 1.

A. PROGRAM FUNCTION CONTINUOUS REGRESSION METHOD VIA NEURAL NETWORK

It is prerequisite for achieving keyword segment marking that fitting the discrete, non-conductible binary program function into a continuous derivable function through neural network, and the neural network structure is vital while fitting. The construction of neural network includes three aspects: input modeling, network structure model, and output modeling. This paper selects the real-valued vector and one-hot vector input modeling method respectively to model the program input according to the characteristics of binary program input, and implements the CNN-based classification network structure and regression network structure. On this basis, we build relevant network models, which are compared through experiments. Additionally, the neural network

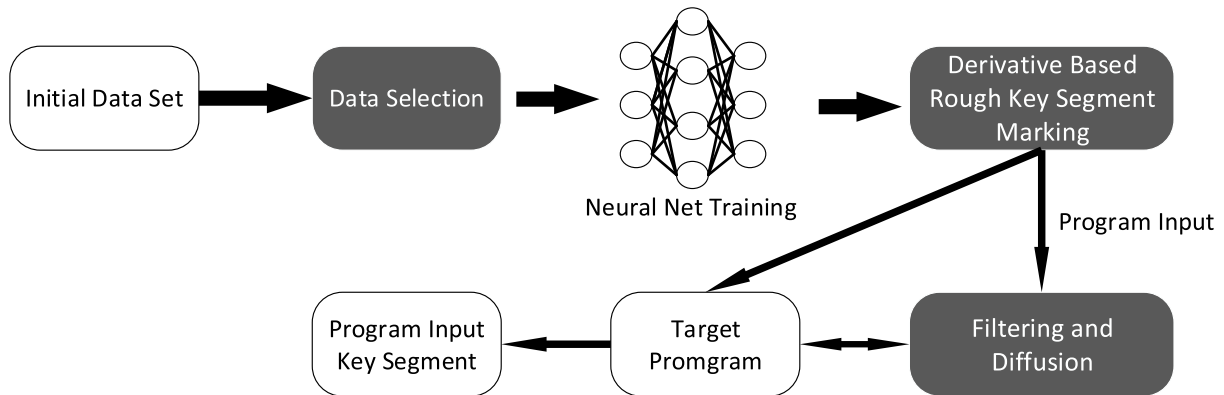


FIGURE 1. The overall design of NeuralTaint.

structure with high fitting efficiency and key segment marking accuracy is selected.

B. DERIVATIVE BASED ROUGH KEY SEGMENT MARKING

For $\vec{y} = F(\vec{x}, \theta)$, the larger the derivative of the program input component, the greater influence on the execution of program function f , and the greater the probability to be a key segment. Based on this principle, the position of the key segment is initially determined by the derivative of program input. However, the fitting rate of neural network usually cannot reach 100%, and the key segment marked directly through derivative information is not accuracy. We call such key segment as rough key segment and achieve accurate key segment marking by filtering and diffusion algorithm 9.

C. FILTERING AND DIFFUSION ALGORITHM

In order to implement an accurate key segment marking method, it is necessary to running the program practically with different input. In this paper, we run the program and monitor the execution of function f by changing the content of rough key segment component, and filter the rough key segment by filtering algorithm. Meanwhile, the key segment marked through the derivative information is usually in byte, and the key segment may be consecutive multi-bytes. Thus, in order to achieve complete and accurate key segment marking, we vary the adjacent bytes of the filtered key segment and further filter those position, which is the diffusion algorithm.

Figure 2 shows the overall process of key segment marking. The yellow block is the key segment of the program input, the blue shadow block is the rough key segment marked by fitted function's derivative information, the green shadow block is the key segment selected by the filtering algorithm and the red shadow block is the final key segment marked by the diffusion algorithm.

IV. IMPLEMENTATION

A. DATASET CONSTRUCTION

Besides the structure of neural networks, the training datasets could also affect the fitting result greatly. The training

datasets include input data for network with corresponding label data, which refers to program input and the execution times of program function. This section introduces the approaches for collecting network input data and constructing related label data. Additionally, the method of filtering and preprocessing data-set is also elaborated thoroughly.

1) DATASET ACQUISITION

Network training depends on training data, thus requiring a huge amount of program input. In this paper, we leverage fuzzing techniques to obtain training data. As for labels, we leverage the binary instrumentation tool to record the execution times of target function.

2) DATASET FILTERING

Many program inputs generated by fuzzing tool cannot trigger any target function, which are regarded as noise data in this paper. The number of noise data should be selected appropriately since excessive noise data could result in low fitting rate, while insufficient noise data might cause overfitting. Therefore, the noise data should be added appropriately in the datasets. In this paper, noise data and non-noise data are separated according to the label, and the training datasets are constructed according to a certain proportion of non-noise data and noise data.

3) DATASET PREPROCESSING

Neural network input requires a fixed length, while most program inputs have variable length. Thus, we select the program input within a predetermined maximum length as dataset, and fill the part less than the set length with specific characters before modeling. In this paper, we fill the real-valued vector with -1 , while adding a new dimension for one-hot vector. More specific data modeling techniques are described in neural network construction part in next section.

B. NEURAL NETWORK CONSTRUCTION

The most essential issue in the key segment marking based on neural network is the construction of neural networks.

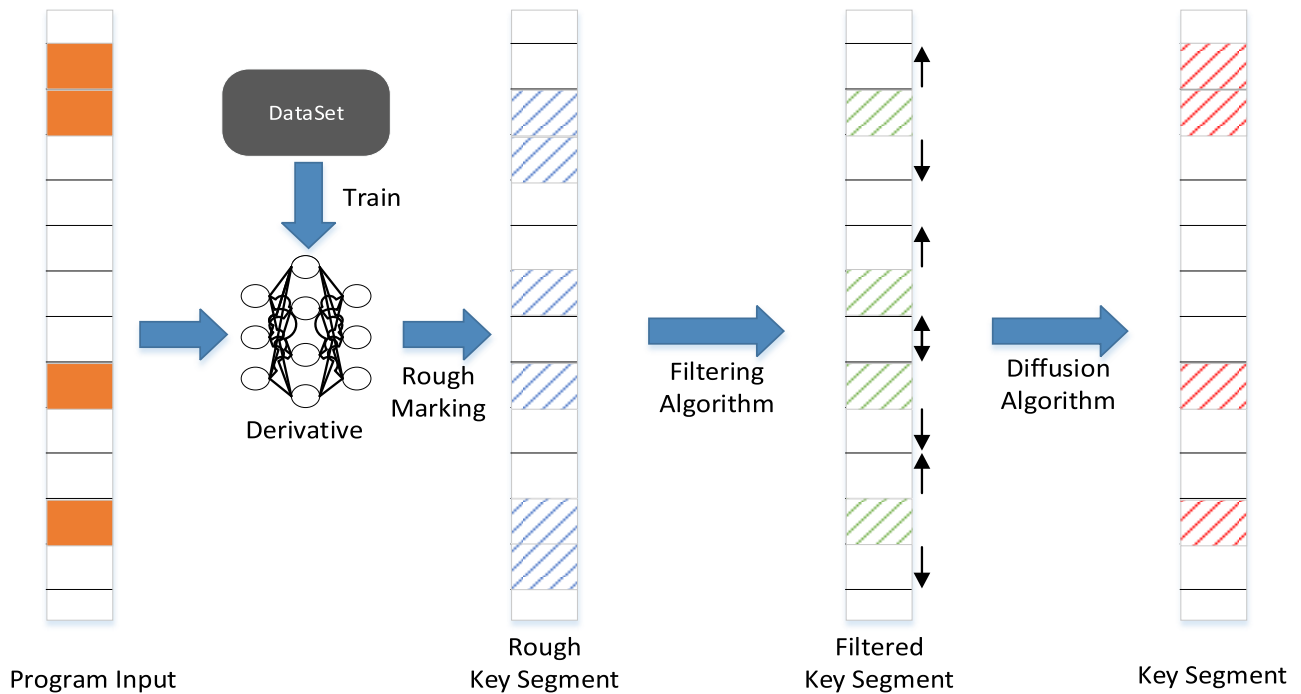


FIGURE 2. The overall process of key segment marking.

Only the neural network with high fitting accuracy could achieve accurate key segment marking. Three main problems are needed to solve neural network constructing, which are program input modeling, network selection, and output modeling.

1) PROGRAM INPUT MODELING

Program input modeling converses the program input into numeral matrix which is acceptable to the neural network. In general, we introduce two methods to model the program input, real-valued vector modeling and one-hot vector modeling. The real-valued vector modeling maps the program input from the integer set $\{0x00, 0x01, \dots, 0xFF\}^n$ to $\{\mathbb{R}\}^n$, converting a program input into an $n \times 1$ float vector. Such input modeling may lead to a faster neural network training speed due to the fewer dimension that involves fewer training parameters of neural network. However, it may suffers not high training accuracy as a result of low dimensions and the insufficient information. One-hot modeling converts the program input from the integer set $\{0x00, 0x01, \dots, 0xFF\}^n$ to a 256-dimensional vector $\{[1, 0, \dots, 0]^T, [0, 1, \dots, 0]^T, \dots, [0, 0, \dots, 1]^T\}^n$ [15]. This modeling technique has higher dimensions, thus containing more information, and leading to higher training accuracy. The derivative also contains more information, which can effectively reduce the difficulty of subsequent analysis. However, the higher dimension requires much more related parameters for network, resulting in time consumption and space redundancy for training. Additionally, besides rich useful information, higher dimensions also introduce redundant

information, causing computing resources wasting. In this paper, two kinds of input modeling for neural networks are implemented and relevant experiments are designed to compare the advantages and disadvantages in training.

2) NETWORK STRUCTURE

The fitting techniques for classification and regression network are introduced in Sections 2. The differences between two different fitting methods are mainly reflected in the backend of network structure, including the output layer, the activation function and corresponding loss function. The backend of the classification network usually combines fully connected layer (FC layer), softmax activation function with the cross entropy loss function or the FC layer [16], sigmoid activation function with the binary cross entropy loss function [17]. These two methods are commonly used in image classification and other issues. The softmax activation function usually solves multiple types problems [18], that categories are mutually exclusive, while sigmoid activation function allows multiple types of coexistence [19]; In this paper, we analysis the effect of program input on the execution of program functions. If we explore multiple functions of the program, the input may trigger multiple functional areas of the program at the same time. Hence, the sigmoid method is adopted for modeling on regression since the neural network output needs multiple types of coexistence. The regression network backend structure includes the FC layer, Relu activation function and MSE loss function [20] or the FC layer, tanh activation function and the MSE loss function [21]. Since the program input affects the program

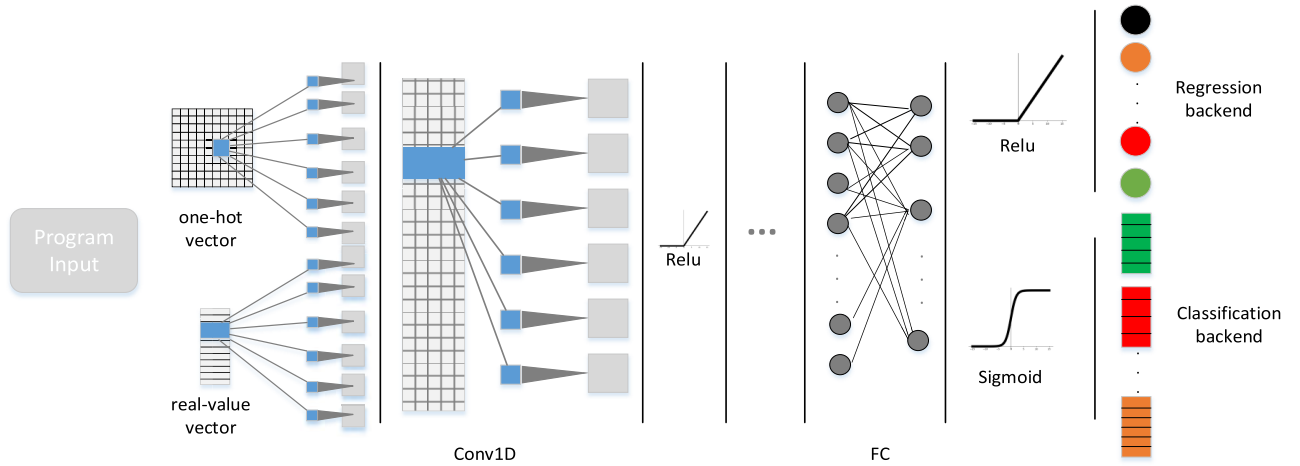


FIGURE 3. The CNN structure of two input and two output modeling method.

function with a positive number, we select the combination of Dense, Relu and MSE for regression network structure.

Currently used network structures mainly include forward neural network [22], recurrent neural network(RNN) [23], and convolutional neural network(CNN) [24]. With the expanding scale of software, the size of program input has gradually increased, and the program input of some commercial software has reached megabytes. The parameters to be trained of the forward neural network increase exponentially with the increase of network input, making it unable to deal with long sequence input. Existing studies have implicated the decline fitting performance of RNN when processing input sequence with length over 150 [25]. And we find that general program input contains significantly more tokens than a natural language sentence, and thus it may be inappropriate for RNN to deal with such long sequence. The CNN with spatial multiplexing method is not sensitive to the input length, which is capable for processing long sequence input effectively [26]. Therefore, this paper uses CNN as main structure of data processing.

Generally speaking, CNN usually join the pooling layer while processing images, which narrows the dimensions of processed matrix by certain algorithms, thereby reducing the number of neural network parameters [27]. The neural network in this paper mainly processes the program input. Since the trigger of some functions requires specific input, the related information might be lost and the fitting performance of neural network with pooling layer might be affected. We adopt multiple 2D convolution layer without pooling layer as our neural network structure.

3) NETWORK OUTPUT MODELING

As for the classification network, the output needs to be modeled in one-hot vector $\{0, 1\}^{\sum_{i=1}^n \max(y_i)}$. We convert each output component y_i into a one-hot vector with length of $\max(y_i)$, then concatenate these vectors into one output vector. The one-hot vector modeling method limits the maximum value

of each output component, which restricts the label data. As for regression networks, we convert each output component to a real-valued vector. Such output modeling method is widely applied due to the simple process compared with that of classification network. However, classification network usually enjoys higher fitting accuracy due to the more suitable backend network layer and activation function [19]. In order to verify validity of two output modeling methods, we construct the neural network with both output modeling and design related experiment to compare the advantage and disadvantage of these network.

In this paper, we design neural networks shown in Figure 3, the classification network based on real-valued vector modeling and regression network based on one-hot vector modeling respectively. In the experimental part(Section 5), a concrete demonstration and description of the performance of these networks will be given thoroughly.

C. DERIVATIVE-BASED KEY SEGMENT ROUGH MARKING METHOD

This section mainly focuses on the method of marking key segment with the derivative information of fitted function. And two problems are fully discussed: how to compare the derivative according to the characteristics of the two input modeling methods and how to achieve rough key segment marking with the multiple fitted neural networks by statistical analysis.

The fitted function $\bar{y} = F_p(\vec{x}, \theta)$, where θ represents the parameters of fitted neural network. And we got the derivative $G_i = \partial y^i / \partial \vec{x}$, which shows the effect of input $\vec{x}$ on output y^i , where y^i represents the i -th output component of $\bar{y}$. As for real-valued input modeling method, $G_i \in \{g_j | 0 < j < n, g_j \in \mathbb{R}\}$, where n represents the length of input. As for one-hot vector modeling method, $G_i \in \{g_j | 0 < j < n, g_j \in \{\mathbb{R}\}^{256}\}$. To illustrate the impact of input changes on the output effectively, we take the absolute value $G_{Ri} = |G_i|$ for every G_i .

For most program inputs, their length m is usually shorter than the neural network input length n . Since the $F_p(\vec{x}, \theta)$ describes the entire training dataset, the position of maximum derivative may be greater than m . In this case, we only analyze the derivative of the program input within length m . For real-valued vector modeling, we have $G_{Ri} \in \{g_j | 0 \leq j < m, g_j \in \mathbb{R}, 0 \leq g_j\}$. For sake of subsequent analysis and calculation, the elements of G_{Ri} is normalized, that is $g_{nj} = g_j / \max(G_{Ri})$ for every component of G_{Ri} . Then the derivative to be analyzed is $G_{Ni} \in \{0 < j < m, g_{nj} \in \mathbb{R}, 0 \leq g_{nj} \leq 1\}$.

For one-hot vector modeling, we also have $G_{Ri} \in \{g_j | 0 \leq j < m, g_j \in \{\mathbb{R}\}^{256}, 0 \leq g_j\}$, and the derivative includes not only the effect of each input component on a particular output, but also the effect of each byte in the input component on the output. For each component g_j , we select the maximum value as the derivative of the component, that is $g_{rj} = \max(g_j)$. Similarly, we normalize these new derivatives, that is $g_{nj} = g_{rj} / \max(G_{Ri})$. Then, the derivative to be analyzed is $G_{Ni} \in \{0 < j < m, g_{nj} \in \mathbb{R}, 0 \leq g_{nj} \leq 1\}$, which shares the same forms with that of real-valued vector modeling.

Since the fitting accuracy of neural network can hardly reach 100%, making it inaccurate for marking key segment through derivative information. Some neural networks have high fitting accuracy in some epochs during the training process, so it is necessary to use these networks with high fitting rate to calculate the derivative to achieve the relatively accurate rough key segment. We select M fitted functions $\vec{y} = F_p(\vec{x}, \theta_i)$ with different fitting rate α_i , and get M derivative $G_{Nai} \in \{g_{nj} | 0 < j < m, g_{nj} \in \mathbb{R}, 0 \leq g_{nj} \leq 1\}$ of each output component y_i separately. And the derivative of input component j is $g_{Mj} = \frac{1}{M} \sum_{i=1}^M g_{nj} \times \alpha_i$. This expression indicates that the higher the fitting accuracy, the higher the importance of corresponding derivative. The derivative to be analyzed could be expressed as $G_{Ni} \in \{g_{Mj} | 0 < j < m, g_{Mj} \in \mathbb{R}, 0 \leq g_{Mj} \leq 1\}$, and as g_{Mj} becomes larger, the j -th input component affects more on execution of program function y_i .

D. FILTERING AND DIFFUSION ALGORITHM

This section introduces the evaluation method for the rough position of key segment. And the key segment is further filtered and diffused to achieve complete and accurate key segment marking.

1) EVALUATION METHOD OF ROUGH POSITION

Assuming a program function triggered by input $\vec{x}$ is executed y_i times. If the changes of k -th component x_k result in the decline of y_i , conclusion could be drawn that the k -th component has an impact on y_i . And there are two criteria for measuring the extent of this impact, which are the changed quantity of y_i and x_j . If the changed byte x'_j of x_j results in increasing the change of y_i , it implies that x'_j triggers some branches that away from program function or triggers other functions that are mutually exclusive with y_i , which indicates that x'_j is less valuable as the key segment. Similarly, as for input component x_j , when traversing in $[0, 0xFF]$, the more bytes it affects y_i , the easier to trigger the program function by

these bytes, which shows that this component has little value to trigger the function. Therefore, as for input component x_j , it could be represented as x_j^k where $k \in [0, 0xFF]$, and the impacts on program function y_i could be expressed as y_i^k . The byte's value as key segment could be evaluated as in (6), and the higher the score, the greater the value as a key segment.

$$S = \frac{\sum_{k=0}^{255} \frac{\text{sgn}(|y_i^k - y_i|)}{e^{|y_i^k - y_i|}}}{\sum_{k=0}^{255} \text{sgn}(|y_i^k - y_i|)} \times \frac{1}{\sum_{k=0}^{255} \text{sgn}(|y_i^k - y_i|)} \quad (6)$$

The Algorithm 1 describes the evaluation method of k -th component of input $\vec{x}$.

Algorithm 1 The Scoring Algorithm of Input Position k

Input: $input \leftarrow$ program input
 $ChangeList \leftarrow$ variable bytes of program input
 $k \leftarrow$ the position of input component
 $y_j \leftarrow$ output's j -th component

Output: $Score \leftarrow$ value of position k

FLOAT GetScore(input, ChangeList, k, y_j):

- 1: $Score \leftarrow 0$
- 2: **for** $ChangeByte \in ChangeList$ **do**
- 3: $input[k] \leftarrow ChangeByte$
- 4: $y'_j \leftarrow F_p(input, \theta)$
- 5: **if** $\text{abs}(y'_j - y_j) > 0$ **then**
- 6: $Count \leftarrow Count + 1$
- 7: $Score \leftarrow Score + e^{-\text{abs}(y'_j - y_j)}$
- 8: **end if**
- 9: **end for**
- 10: $Score \leftarrow Score / Count^2$
- 11: **return** $Score$

2) FILTERING ALGORITHM

According to the value evaluation method, we filter the location of key segment by setting relevant thresholds. As for real-valued vector modeling, the input x_j varies from 0 to 0xFF. The derivative of one-hot vector modeling contains much more information compared with real-valued vector modeling, and the $g_j \in \{\mathbb{R}\}^{256}$ of input x_j contains the impacts of each byte of $[0, 0xFF]$ on output. Hence, the range of variation of the input component x_j is the byte corresponding to the larger derivative g_j . The algorithm for filtering rough marked key segment is shown in Algorithm 2.

3) DIFFUSION ALGORITHM

The diffusion algorithm completes the key segment marking based on the filtered position of key segment. The filtering algorithm is performed on the adjacent position of filtered key segment until the component score less than the threshold or reach the boundary of the program input. The diffusion algorithm of component k is shown as in Algorithm 3.

V. EXPERIMENT

In this section, we design and implement key segment marking method as part of NeuralTaint, a control flow taint

Algorithm 2 Filtering Algorithm of Key Segment

Input: $input \leftarrow$ program input
 $gradient \leftarrow$ the derivative of y_j
 $k \leftarrow$ the position of input component
 $y_j \leftarrow$ output's j-th component
 $Threshold \leftarrow$ the preset minimum score of key segment

Output: whether the k-th component of program input is key segment

BOOL isKey($input, gradient, k, y_j, Threshold$):

- 1: **if** isOneHotModel **then**
- 2: $ChangeList \leftarrow topk(gradient)$
- 3: $\# topk() \leftarrow$ the largest k of the parameter
- 4: **else**
- 5: $ChangeList \leftarrow [0x00, 0xFF]$
- 6: **end if**
- 7: $Score \leftarrow GetScore(ChangeList, k, y_j, input)$
- 8: **if** $Score > Threshold$ **then**
- 9: **return true**
- 10: **else**
- 11: **return false**
- 12: **end if**

analysis tool. And design two main experiments to verify the validity of NeuralTaint. The first experiment is to select the best parameters of the network structure. We build several CNN structure with different input and output modeling, and test on one real-world software to select the most suitable CNN structure, input modeling and output modeling method for NeuralTaint. The second experiment is to verify whether NeuralTaint is outperform than traditional taint analysis tool. We select 7 real-world program and 50 program inputs each to mark the key segment of the predetermined function and determine the performance by comparing the marking accuracy.

A. CNN STRUCTURE EXPERIMENT**1) TARGET**

We select rdjpgcom as target program, which is the sample program of libjpeg used to read and parser comment information of jpeg files.

2) INPUT DATA PREPARATION

We leverage AFL [28] to perform fuzzing test for this program, and collect program input every 20 times execution. Moreover, in order to ensure the randomness of the noise data, we start five AFL engines to perform fuzzy test respectively. Each fuzzy test is performed for one hour, and a total of 500k program inputs are collected. The following data collection method in this paper is similar to this method.

3) LABEL DATA PREPARATION

We select the functions triggered by the switch-case structure as target point in rdjpgcom. This kind of program structure may lead to control flow dependency, which could be

Algorithm 3 Diffusion Algorithm

Input: $input \leftarrow$ program input
 $gradient \leftarrow$ the derivative of y_j
 $k \leftarrow$ position of key segment with rough location
 $y_j \leftarrow$ output's j-th component
 $T \leftarrow$ the preset minimum score of key segment($Threshold$)

Output: KeySegment $\leftarrow$ the key segment of the program input component k

LIST Diffusion($input, gradient, k, y_j, Thresholds$):

- 1: $\#BackWard$ Spred Analysis
- 2: $position \leftarrow k$
- 3: **while** 1 **do**
- 4: **if** $position > len(input)$ **then**
- 5: **break**
- 6: **end if**
- 7: **if not** isKey($gradient[position], position, y_j, input, T$) **then**
- 8: **break**
- 9: **end if**
- 10: $KeyAreaBackWard.append(position)$
- 11: $position \leftarrow position + 1$
- 12: **end while**
- 13: $position \leftarrow k$
- 14: $\#Forward$ Spread Analysis
- 15: **while** 1 **do**
- 16: $position \leftarrow position - 1$
- 17: **if** $position < 0$ **then**
- 18: **break**
- 19: **end if**
- 20: **if not** isKey($gradient[position], position, y_j, input, T$) **then**
- 21: **break**
- 22: **end if**
- 23: $KeyAreaForWard.append(position)$
- 24: **end while**
- 25: $KeySegment \leftarrow reverse(KeyAreaForWard) + KeyAreaBackWard$
- 26: **return** KeySegment

analyzed to verify the effectiveness and applicability of our method. This paper monitors the execution of function in the target programs with pintool [29], recording and analyzing the outputs of 500k program inputs.

rdjpgcom is compiled under 64-bit operating system. We choose the position of 0x400A7A, 0x400B28 and 0x400A64, which are analyzed for the image information output, variable analysis and annotation analysis. The execution times for these three positions are regarded as the network output.

4) NEURAL NETWORK INPUT LENGTH

By sorting the lengths of all program inputs, we search for the data that could cover 80% of the entire input and has

TABLE 1. All neural network highest fitting accuracy.

Fitting Accuracy	RR	RC	OR	OC
3 layers small kernel	0.813	0.848	0.932	0.916
3 layers huge kernel	0.786	0.842	0.929	0.916
9 layers small kernel	0.838	0.858	0.935	0.935
9 layers huge kernel	0.815	0.871	0.925	0.905

3 layers stands for the neural network has 3 convolutional layers. Small kernel stands for each convolutional layer uses small convolution kernel and huge kernel stands for each convolutional layer uses huge convolution kernel.

RR stands for real-valued vector input regression neural network, RC stands for real-valued vector input classification neural network, OR stands for one-hot vector input regression neural network, OC stands for one-hot vector input classification neural network.

the smallest difference between the maximum input length with the minimum input length. We select the maximum input length of that part of data to be the neural network length input.

5) NOISE DATA FILTERING

In this paper, we regard the data that result in all zero output as noise data. Mixing non-noise data and noise data in a 3:1 ratio. The following data preparation method is similar to this method.

This paper finally selects a network with the input length 400. The maximum output of 0x400A7A is 8, within 2 for 0x400B28 and 0x400A64 respectively. The final selected data set has a total of 15,000 data, of which 15% is selected as verification data and the rest is used as training data.

6) MODEL PREPARATION

We construct 16 networks with two different input modeling methods, two different network types and four different CNN constructions. To compare the fitting speed, fitting accuracy and key segment marking accuracy. Small convolution kernel refers to 3 layers with 300 filters of size 3,2 and stride 3,2,1 respectively. Huge convolution kernel refers to 9 to 12 layers with 300 filters of size 9,10,11,12 and stride 3,4,5 respectively.

When training network, we set the batch size as 128, and the iteration for 100 rounds. The training accuracy is the ratio of the amount of data within less than 0.5 difference of network output and real output to total amount of training data. We select the network with fitting accuracy more than 80% to derive and mark key segment, and the training data is reorganized after every training epoch.

7) EXPERIMENTAL ENVIRONMENT

This paper leverages ubuntu 16.04 as test system within tensorflow 1.11.0 and graphics card is GTX1080ti. CPU is 6800k @3.8GHz, and memory is 32G.

8) EXPERIMENT RESULT

After 100 rounds iterations, the highest fitting accuracy is shown as in Table 1.

TABLE 2. All network training time of 100 rounds.

Training Time(second)	RR	RC	OR	OC
3 layers small kernel	802.4	542.4	2321.1	1783.9
3 layers huge kernel	768.4	523.1	2236.7	1765.5
9 layers small kernel	900.5	583.4	2457.5	1800.9
9 layers huge kernel	702.4	621.4	2122.0	1798.2

TABLE 3. All network key segment marking accuracy.

Key Segment Marking Accuracy	RR	RC	OR	OC
3 layers small kernel	0.879	0.859	0.956	0.925
	0.869	0.848	0.952	0.913
	0.860	0.823	0.954	0.903
3 layers huge kernel	0.842	0.869	0.987	0.947
	0.823	0.838	0.980	0.930
	0.813	0.833	0.973	0.933
9 layers small kernel	0.908	0.879	0.978	0.938
	0.899	0.858	0.974	0.924
	0.890	0.843	0.976	0.914
9 layers huge kernel	0.888	0.878	0.974	0.954
	0.879	0.869	0.964	0.944
	0.870	0.860	0.962	0.932

The time consumption of 100 rounds fitting is shown as Table 2.

We select 50 programs input without noise data to mark the key segment, and determine the correctness through manual debugging. The analyzing accuracy of position of 0x400A7A, 0x400B28, and 0x400A64 is shown in Table 3.

9) RESULT ANALYSIS

In this experiment, we leverage the program structure that could trigger the control flow dependency as analyzing targets, and carry out comparative experiments for different neural networks. The experimental results show that NeuralTaint could realize the key segment marking of the control flow dependent program structure, which is capable of overcoming the problem of control flow dependence in the traditional taint analysis, and analyzing multiple different program functions at the same time.

In terms of training time, the one-hot modeling method takes longer time as a result of much more training parameters. However, the classification networks containing only one FC layer network during the output consume less time than networks with two FC layers. Meanwhile, although network parameters such as depths and convolution kernel sizes differ with networks, their parameters is much less than FC layers', which means that these layers have little effect on training time.

In terms of fitting effects, one-hot modeling with high-dimensions contains more input information, thus enjoying a higher fitting rate. The classification network and the fitting network share analogous fitting rates since the value of each label in the training data is not very large and each output has more training data. However, the classification network has limitations on the range of network output and the modeling process is cumbersome. Moreover, excessive classifications may lead to overfull backend parameters

TABLE 4. Experimental programs and functional location.

Program	Function Address	Target Function Description
File(libmagic)	libmagic_base +0x12862	This position is <i>json_parse_string</i> function called by <i>json_parse</i> function. The number of calls indicates the amount of strings from json file. Triggering the function requires a program structure that controls the flow dependency.
xmlwf(libexpat)	libexpat_base +0x8A70	This position is case XML_TOK_START_TAG_WITH_ATTS from <i>docontent</i> used to parse tag header of xml and the number of calls indicates the amount of tag header to be parsed. Triggering the function requires a program structure that controls the flow dependency.
jhead	0x406090	This position is entry of function <i>process_DHT</i> which is used to evaluate the quality of picture. Triggering the function requires a program structure that controls the flow dependency.
readelf	0x415BE0	This position is default in section header of function <i>process_section_headers</i> . The number of execution times represents the amount of section from input. Triggering the function requires a program structure that controls the flow dependency.
tiffinfo -D -c -d	0x401795	This position is the location of showing the line number of the tiff image in the function <i>ShowStrip</i> , the calling times represents the number of lines in the tiff image.
size	0x4035B8	This position is the location of symbolic technology in function <i>calculate_comm_size</i> . The calling times represents the amount of symbol of elf. Triggering the function is not directly related to the program input.
mupdf	0x5120CA	This position is the entry of <i>pdf_load_object</i> , and the calling times represents the amount of obj in pdf. Triggering the function is not directly related to the program input.

of network with overfitting problems. Meanwhile, the experimental results show that CNN with 9 convolution layers and small convolution kernel could reach higher accuracy. The deep small convolution kernel networks not only need less training parameters, but also can represent more characteristics of network input.

In terms of the key segment accuracy, we found it positively correlated with the network fitting rate. The higher the network fitting rate, the higher the marking accuracy of the key segment. The backend function of the classification network is Sigmoid function, and output range is between (0, 1). Due to the lack of capability for computer floating-point representation, the output will be represented by 1.0 at the late of training, and the derivative of the output component is 0, which resulting in the accuracy less than that of regression network. Since the number of data at 0x400A64 and the 0x400B28 are less than that at 0x400A7A, the accuracy at 0x400A64 and 0x400B28 of key segment analysis are lower than that at 0x400A7A.

Due to the relationship between keyword segment marking accuracy and network fitting rate, and taking the difficulty of network model construction, fitting accuracy and anal-

TABLE 5. Training dataset.

Programs	Number of Noise Data	Number of Non-noise Data	Input Length
file	79380	84000	247
xmlwf	102893	65000	256
jhead	90001	30000	382
readelf	88170	187000	497
tiffinfo	11401	114000	497
size	11401	114000	497
mupdf	108857	256000	347

TABLE 6. Time consumption and fitting accuracy.

Programs	Training Time(second)	Fitting Accuracy
file	8220	0.995
xmlwf	8421	0.985
jhead	9177	0.974
readelf	7019	0.998
tiffinfo	4411	0.990
size	10201	0.997
mupdf	10096	0.963

ysis accuracy into consideration, we finally select regression network of deep small convolution kernel based on one-hot input modeling as NeuralTaint basic architecture. The following experiments in this paper all use this network structure.

B. EXPERIMENTS ON REAL PROGRAMS AND COMPARISONS WITH TRADITIONAL TOOLS

In this section, we select real-word programs on linux platform, and choose one slice of programs as target area. The target file formats include jpg, json, xml, elf and pdf files. We select 50 inputs for each program as testing sample, and mark the key segment. For comparison with other tools, we analyze these program input with BitBlaze [30] and S2E [31].

BitBlaze is a dynamic taint analysis tool developed by Song and Dawn [30] that implements taint analysis in byte-level using a system-level virtualization platform. S2E is a dynamic symbol execution tool developed by Chipounov et al. [31] that uses a system-level virtualization platform qemu with klee as symbolic execution backend. Both tools are popular tools in dynamic analysis research.

Table 4 lists the experiment programs and target functions to be analyzed.

Each program is fuzzed through AFL for one hour. We use pintool to label the program input and then filter the datasets. The training dataset for each program is shown in Table 5.

After 100 iterations, the consumption of time and fitting rate is listed in Table 6.

We select 50 program inputs for each program to analyze the accuracy of key segment. And the accuracy calculation method is in (7). We leverage BitBlaze and S2E for analyzing the inputs, extracting key segments, and verifying accuracy. The accuracy of the experimental results is listed as

TABLE 7. Comparison experiment result.

Programs	NeuralTaint	BitBlaze	S2E
file	0.998	0.546	0.654
xmlwf	0.970	0.452	0.598
jhead	0.957	0.698	0.876
readelf	0.986	0.645	0.786
tiffinfo	0.930	0.425	0.543
size	0.972	0.648	0.754
mupdf	0.959	0.345	0.422

in Table 7.

$$Accuracy = \frac{\text{correct marked key segment}}{\text{total key segment}} \quad (7)$$

With comparison with traditional dynamic analysis methods, NeuralTaint has higher accuracy in the analysis of taint related to control flow, while BitBlaze and S2E fail to analyze due to control flow dependency problems, and only several specific segments that affect the critical position of the control flow can be analyzed by these tools.

Compared with traditional analysis methods, NeuralTaint can effectively overcome the problems of control flow dependence and symbolic address when analyzing controlling flow-related taint, thus having higher accuracy than traditional analysis methods.

VI. RELATED WORK

Related research on program analysis and neural networks recently develops rapidly. Dawn Song [32] firstly realizes the recognition of function boundaries on machine-code level through neural networks. This method is the first to apply artificial intelligence techniques to the field of software vulnerability analysis. It models the machine code of program in bytes and implements the mapping of machine code to function boundaries with RNN network. However, as a result of the limitation of the long sequence of the RNN network, the method still has a certain gap with the traditional program static analysis method in the actual accuracy. EKLAVYA [33] implements the identification of function parameters based on deep learning. The main idea uses word2vec to realize the modeling of assembly code, and selects GRU network to realize the recognition of the number and type of function parameters based on assembly code. NEUZZ [34] uses gradient-guide input generation method to increase the efficiency of fuzzing process based on surrogate neural network. Such research works indicate that neural network can learn the relationship between network input and output without any program internal information.

1) DYNAMIC TAINT ANALYSIS

Taint analysis can track critical bytes in the input that affect the actual control flow of program execution. Currently, dynamic taint analysis tools are mainly divided into application-aware analysis tools and system-wide analysis tools. Application-aware analysis tools typically leverage a binary instrumentation tool to analyze the relationship

between the execution of critical program segments with the program input during operation. Application-aware taint methods analyze at a high speed, which is often used in vulnerability discovering. TaintScope [35] can identify security-sensitive bytes that affects system call and library call and focus on mutating these bytes. Dowser [36] targets buffer boundary violations. It uses light-weight taint analysis to locate input bytes that affect the array index, and mutate these bytes to explore vulnerability. BORG [37] targets buffer over-read vulnerability by selecting buffer accesses that could lead to an over-read. VUzzer [38] uses application-aware taint analysis to locate the position of magic bytes in program input, and then assigned these magic bytes to fixed positions in the input. Angora [39] applies byte-level taint tracking to get the byte offsets in the input that flow into each conditional statement, and then mutates these bytes to satisfy the condition for the unexplored branch. Although application-aware taint analysis could work at a high speed, these methods cannot analyze system kernel and code fragments with system calls, resulting lower analyzing accuracy. The application-aware taint tools usually depend on the virtualization platform qemu with fine-grained taint propagation, which can effectively analyze the taint spreading process input into the system kernel. This method has higher accuracy, but suffers from slower speed and more complex implementation, thus not widely used. BitBlaze [30] is designed to enable accurate analysis, provide an extensible architecture, and combine static and dynamic taint analysis as well as program verification techniques to satisfy the common needs of security applications. PANDA [40] adopts the translation-execution mechanism from QEMU [41]. The basic block before translation is recorded, and researchers analyze the translated basic block with taint. The process of execution and analysis are isolated, which not only improves the analysis speed but also avoids the impacts of taint propagation algorithm on target program. However, neither application-aware nor system-wide taint analysis techniques could effectively solve the problem of control flow dependency, taint address, overtainting and undertainting, thus having certain restrictions on the real-world programs.

2) DYNAMIC SYMBOLIC EXECUTION

Dynamic symbolic execution represents predicates in the program as a set of constraints in the first-order logic formula, then solve these constraints by calling a solver to obtain the input that can trigger certain branches. Similar with taint analysis method, dynamic symbolic execution is mainly divided into application-aware methods and system-wide methods. Application-aware method is usually applied to assisted fuzzing test to improve path coverage with fast analysis speed and low accuracy. FuzzWin [42] uses application-aware symbolic execution to generate inputs that can trigger target program execution branch in fuzzing. SYMFUZZ [43] leverages symbolic execution on a given seed pair to detect dependencies among the bit positions of program input. Driller [44] uses selective concolic execution to generate test case when

the fuzzer gets stuck. QSYM [45] integrates the symbolic execution with the native execution using dynamic binary translation. application-aware dynamic symbolic execution suffers from not high accuracy due to the inability to obtain system kernel information, while system-wide symbolic execution tools are hardly used as a result of complicated structure and difficult development. S2E [31] is a system-wide platform for analyzing the properties and behavior of software systems based on QEMU, it uses selective symbolic execution and relaxed execution consistency models to scale to large software. Current related research mainly focuses on solving the problem of state explosion, but still unable to solve the control flow dependence and symbol pointer. Therefore, dynamic symbolic execution still has certain obstacles for real-world program analysis. Dynamic taint analysis and symbolic execution relies on heavy-weight program analysis techniques with nontrivial instrumentation overheads. On the contrary, NeuralTaint locate key segment in program input from neural network without any expensive program analysis techniques.

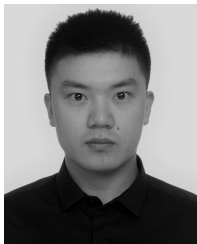
VII. CONCLUSION

This paper designs and implements a control flow taint analysis tool NeuralTaint based on neural network. Based on the function derivative principle, the tool leverages the neural network to fit the execution of program, and then determines the key segment that affects the execution of the program function, and finally implements the key segment identification by filtering and diffusion algorithm. This paper carries on the experiments and selects the network structure that is suitable for key segment marking. By comparing with traditional analysis methods on real-world programs, NeuralTaint can effectively overcome the shortcomings of traditional analysis methods with much higher accuracy. In summary, NeuralTaint has developed a different idea for taint analysis methods, and the network comparison experiments provide a large amount of data foundation for subsequent related work.

REFERENCES

- [1] E. J. Schwartz, T. Avgerinos, and D. Brumley, "All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 317–331.
- [2] M. Ganai, D. Lee, and A. Gupta, "DTAM: Dynamic taint analysis of multi-threaded programs for relevance," in *Proc. ACM SIGSOFT Int. Symp. Found. Softw. Eng.* 2012, pp. 1–11.
- [3] C. Csallner, N. Tillmann, and Y. Smaragdakis, "DySy: Dynamic symbolic execution for invariant inference," in *Proc. ACM/IEEE Int. Conf. Softw. Eng.*, vol. 31, no. 31, May 2008, pp. 301–313.
- [4] T. Xie, N. Tillmann, J. de Halleux, and W. Schulte, "Fitness-guided path exploration in dynamic symbolic execution," in *Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun./Jul. 2009, pp. 359–368.
- [5] B. Shuai, M. Li, H. Li, Q. Zhang, and C. Tang, "Software vulnerability detection using genetic algorithm and dynamic taint analysis," in *Proc. Int. Conf. Consum. Electron., Commun. Netw.*, Nov. 2013, pp. 589–593.
- [6] M. Papadakis and N. Malevris, "Automatic mutation test case generation via dynamic symbolic execution," in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, Nov. 2010, pp. 121–130.
- [7] L. Deng and D. Yu, "Deep learning: Methods and applications," *Found. Trends Signal Process.*, vol. 7, no. 3, pp. 197–387, 2014.
- [8] G. S. Watson, "Smooth regression analysis," *Sankhyā, Indian J. Statist. A*, vol. 26, no. 4, pp. 359–372, 1964.
- [9] D. M. Bates and D. G. Watts, "Nonlinear regression analysis and its applications," *Technometrics*, vol. 32, no. 2, pp. 219–220, 1988.
- [10] A. Cucchiaro, "Applied logistic regression," *Technometrics*, vol. 34, no. 3, p. 2, 2012.
- [11] C. Mood, "Logistic regression: Why we cannot do what we think we can do, and what we can do about it," *Eur. Sociol. Rev.*, vol. 26, no. 1, pp. 67–82, 2010.
- [12] D. F. Specht, "A general regression neural network," *IEEE Trans. Neural Netw.*, vol. 2, no. 6, pp. 568–576, Nov. 1991.
- [13] D. F. Specht, "The general regression neural network-rediscovered," *Neural Netw.*, vol. 6, no. 7, pp. 1033–1034, 1993.
- [14] Y. Ephraim and D. Malah, "Speech enhancement using a minimum mean-square error log-spectral amplitude estimator," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-33, no. 2, pp. 443–445, Apr. 1985.
- [15] W.-C. Chou et al., "Average-case optimized technology mapping of one-hot domino circuits," in *Proc. 4th Int. Symp. Adv. Res. Asynchronous Circuits Syst.*, 1998, pp. 80–91.
- [16] Liu, Weiyang, "Large-margin softmax loss for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1–10.
- [17] L. K. Saul, T. Jaakkola, and M. I. Jordan, "Mean field theory for sigmoid belief networks," *J. Artif. Intell. Res.*, vol. 4, no. 1, pp. 61–76, 1996.
- [18] X. Liang, X. Wang, Z. Lei, S. Liao, and S. Z. Li, "Soft-margin softmax for deep classification," in *Proc. Int. Conf. Neural Inf. Process.*, 2017, pp. 413–421. doi: 10.1007/978-3-319-70096-0_43.
- [19] G. Daqi and J. Yan, "Classification methodologies of multilayer perceptrons with sigmoid activation functions," *Pattern Recognit.* vol. 38, no. 10, pp. 1469–1482, 2005.
- [20] Y. Li and Y. Yuan. (2017). "Convergence analysis of two-layer neural networks with ReLU activation." [Online]. Available: <https://arxiv.org/abs/1705.09886>
- [21] s. Eger, P. Youssef, and I. Gurevych. (2019). "Is it time to swish? Comparing deep learning activation functions across NLP tasks." [Online]. Available: <https://arxiv.org/abs/1901.02671>
- [22] E. P. P. A. Derks, M. S. S. Pastor, and L. M. C. Buydens, "Robustness analysis of radial base function and multi-layered feed-forward neural network models," *Chemometrics Intell. Lab. Syst.*, vol. 28, no. 1, pp. 49–60, 1995.
- [23] M. Lukoševičius and H. Jaeger, "Reservoir computing approaches to recurrent neural network training," *Comput. Sci. Rev.*, vol. 3, no. 13, pp. 127–149, 2009.
- [24] S. C. B. Lo, S. L. A. Lou, J.-S. Lin, M. T. Freedman, M. V. Chien, and S. K. Mun, "Artificial convolution neural network techniques and applications for lung nodule detection," *IEEE Trans. Med. Imag.*, vol. 14, no. 4, pp. 711–718, Dec. 1995.
- [25] T. Mikolov, M. Karafiát, L. Burget, J. Cernocký, and S. Khudanpur, "Recurrent neural network based language model," in *Proc. 11th Annu. Conf. Int. Speech Commun. Assoc.*, 2010, pp. 1–24.
- [26] N. Kalchbrenner, E. Grefenstette, and P. Blunsom. (2014). "A convolutional neural network for modelling sentences." [Online]. Available: <https://arxiv.org/abs/1404.2188>
- [27] A. Giusti, D. C. Ciresan, J. Masci, L. M. Gambardella, and J. Schmidhuber, "Fast image scanning with deep max-pooling convolutional neural networks," in *Proc. IEEE Int. Conf. Image Process.*, Sep. 2013, pp. 4034–4038.
- [28] *American Fuzzy Lop*. Accessed: 2018. [Online]. Available: <http://lcamtuf.coredump.cx/afl/>
- [29] *PinTool*. Accessed: 2018. [Online]. Available: <https://software.intel.com/en-us/articles/pin-a-dynamic-binary-instrumentation-tool>
- [30] D. Song et al., "BitBlaze: A new approach to computer security via binary analysis," in *Proc. Int. Conf. Inf. Syst. Secur.* Berlin, Germany: Springer, 2008, pp. 1–25.
- [31] V. Chipounov, V. Kuznetsov, and G. Candea, "S2E: A platform for in-vivo multi-path analysis of software systems," *ACM SIGARCH Comput. Archit. News*, vol. 39., no. 1, pp. 265–278, 2011.
- [32] E. C. R. Shin, D. Song, and R. Moazzezi, "Recognizing functions in binaries with neural networks," in *Proc. 24th USENIX Secur. Symp. (USENIX Secur.)*, 2015, pp. 611–626.
- [33] Z. L. Chua, S. Shen, P. Saxena, and Z. Liang, "Neural nets can learn function type signatures from binaries," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 99–116.

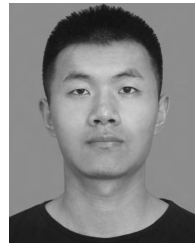
- [34] D. She, K. Pei, D. Epstein, J. Yang, B. Ray, and S. Jana. (2018). "NEUZZ: Efficient fuzzing with neural program smoothing." [Online]. Available: <https://arxiv.org/abs/1807.05620>
- [35] T. Wang, T. Wei, G. Gu, and W. Zou, "TaintScope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 497–512.
- [36] I. Haller, A. Slowinska, M. Neugschwandtner, and H. Bos, "Dowser: A guided fuzzer to find buffer overflow vulnerabilities," in *Proc. 22nd USENIX Secur. Symp.*, 2013, pp. 49–64.
- [37] S. Arzt et al., "Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," *ACM SIGPLAN Notices*, vol. 49, no. 6, pp. 259–269, 2014.
- [38] S. Rawat, V. Jain, A. Kumar, L. Cojocar, and C. Giuffrida, "VUZZer: Application-aware evolutionary fuzzing," in *Proc. NDSS*, vol. 17, pp. 1–14, Feb./Mar. 2017.
- [39] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 711–725.
- [40] S. Dolan-Gavitt, J. Hodosh, P. Hulin, T. Leek, and R. Whelan, "Repeatable reverse engineering with PANDA," in *Proc. 5th Program Protection Reverse Eng. Workshop*, Los Angeles, CA, USA, Dec. 2015, pp. 1–11.
- [41] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. USENIX Annu. Tech. Conf.*, vol. 41, 2005, pp. 1–6.
- [42] S. Lecomte, "Élaboration d'une représentation intermédiaire pour l'exécution concolique et le marquage de données sous windows," APA, Washington, DC, USA, 2014.
- [43] B. S. Pak, "Hybrid fuzz testing: Discovering software bugs via fuzzing and symbolic execution," School Comput. Sci., Carnegie Mellon Univ., Pittsburgh, PA, USA, 2012.
- [44] N. Stephens et al., "Driller: Augmenting fuzzing through selective symbolic execution," in *Proc. NDSS*, vol. 16, 2016, pp. 1–16.
- [45] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "QSYM: A practical concolic execution engine tailored for hybrid fuzzing," in *Proc. 27th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 745–761.



XING ZHANG received the B.Sc. and M.Sc. degrees from the National University and Defense Technology, China, in 2014 and 2016, respectively, where he is currently pursuing the Ph.D. degree. His current research interests include information security, deep learning, and software vulnerabilities.



CHAO FENG received the B.Sc., M.Sc., and Ph.D. degrees from the National University and Defense Technology (NUDT), China, in 2004, 2005, and 2011, respectively, where he is currently an Associate Professor with the College of Electronic Science. His current research interests include protocol analysis, network security, and software vulnerabilities.



RUNHAO LI received the B.Sc. degree from the National University and Defense Technology, China, in 2017, where he is currently pursuing the M.Sc. degree. His current research interests include information security and software vulnerabilities.



JING LEI received the Ph.D. degree from the National University of Defense Technology (NUDT), China, in 2008, where she is currently a Professor of College of Electronic Science. Her current research interests include information security and information coding.



CHAOJING TANG received the Ph.D. degree from the National University of Defense Technology (NUDT), China, in 2003, where he is currently a Professor of College of Electronic Science. His current research interests include information security and software vulnerabilities.

...